Programming Assignment 1

Linux System, Virtual Machines and Command Line Interface

Instructions:

- Assigned date: Thursday January 11th 2023
- Due date: 11:59pm on Monday January 29th 2023
- Maximum points: 100
- This programming assignment must be done individually.
- Please post your questions on the D2L Programming Assignments topic.
- Only the report must be uploaded to D2L. The code must be uploaded to the GitHub repository created for you for this assignment.
- Late submission will be penalized at 20% per late day.

This programming assignment aims to teach you and introduce you to the basics of the Linux Ecosystem, Virtual Machines and the Command Line Interface. The programs that you need to implement as part of this assignment are limited to C++ or Java and to the standard libraries. You can use any computer to complete this assignment (e.g. your laptop running any Operating System).

1 Linux Ecosystem and Virtual Machines (30%)

1. Install on your computer one of the two virtual machine hypervisors: Oracle Virtual Box 7.0.12 or VMware Fusion Player 13 (if you have an Apple M1/M2/M3 computer install VMware Fusion Player 13 or UTM).
2. Download an Ubuntu Server 22.04.3 LTS iso image (if you have Apple M1/M2/M3 computer you might need to download an Ubuntu Server 22.04.3 LTS ARM64 image).
3. Create a Linux virtual machine on the installed hypervisor with the following specifications: 2 cores, 2GiB of RAM, 50GB virtual disk, a custom NAT network interface and a host-only network interface. The custom NAT network interface will be used for Internet communication, while the host-only will be used to connect the host computer to the virtual machine.
4. Install Ubuntu Server on the new virtual machine and configure a username and password.
5. Turn on the firewall, enable SSH access to the virtual machine (with password-based authentication) and open the SSH port on the firewall.

6. Log in from a terminal, opened on the host machine, to the virtual machine using SSH.
7. Create a public-private key pair on your host computer (not on the virtual machine) and enable password-less authentication from the host.

Answer the following questions in your report:

1. Give an example of one or more commands that you can use to retrieve the total number of cores, the total amount of RAM and the total amount of disk storage on a Linux operating system. Add screenshots to the report, showing the output of the commands run inside the virtual machine that you created.
2. What command(s) did you use to enable SSH access to the virtual machine? Provide screenshots of the commands used and the output.
3. What commands did you use to turn on the firewall and to open the SSH port on the firewall? Provide screenshots of the commands used and the output.
4. What is an IP (Internet Protocol) address? What is a TCP/UDP port? What port does SSH use by default? What IP address does your virtual machine have? What commands did you use to retrieve the IP address of your virtual machine? Provide screenshots of the commands used to retrieve the IP address of your virtual machine and the output.
5. What command did you use to generate a private-public key pair? Provide screenshots of the command used and the output.
6. Give an example of a set of commands that you can use to enable passwordless authentication from the host to the virtual machine. Include in the response screenshots of the commands used, the output and proof that you can log in without entering a password.

2 Clean Dataset (20%)

For this part and the remaining parts of the assignment you will need to download 5 datasets. The datasets are ebooks stored as TXT files in folders. In order to download the datasets you have to log in with your DePaul username and password.

Implement a program in C++ or in Java that receives as arguments an input directory and an output directory and that cleans the files from the input directory and writes the cleaned files to the output directory.

The cleaned files must follow the same folder structure as the input files. For example, if the program cleans the file stored at **Dataset1/folder6/document265.txt**, it must store the cleaned file in **CleanedDataset1/folder6/document265.txt**, where **Dataset1** was the input directory and **CleanedDataset1** was the output directory.

The input files are TXT files that contain words separated by separators and by delimiters. In this program, words are defined as any sequence of alphanumerical characters (0-9a-zA-Z). Delimiters are defined as the space, tab and new line characters ('\', '\t', '\n', '\r\n', '\r') and any other character is considered a separator.

The cleaning process that your program needs to implement has to abide by the following rules:

- any '\r' character has to be eliminated;
- any repeating sequence of delimiters must be replaced with the last delimiter in the sequence. For example, if your program encounters "\r\n\r\n", it must replace it with "\n", because '\n' was the last character in the delimiter sequence;
- any separator must be eliminated. For example, if your program encounters "document-01.txt", it must replace it with "document01txt", because '-' and '.' are separator characters (they are not word characters or delimiter characters);

When the program has finished cleaning a file, the output file should contain words composed out of alphanumerical characters separated by only one delimiter and should not contain any separator characters and no repeating delimiters.

For example, if an input file has the following content:

```
EBooks posted since November 2003, with etext numbers OVER #10000, are
filed in a different way.  The year of a release date is no longer part
of the directory path.  The path is based on the etext number (which is
identical to the filename).  The path to the file is made up of single
digits corresponding to all but the last digit in the filename.  For
example an eBook of filename 10234 would be found at:
```

The output file of the corresponding example should be:

```
EBooks posted since November 2003 with etext numbers OVER 10000 are
filed in a different way The year of a release date is no longer part
of the directory path The path is based on the etext number which is
identical to the filename The path to the file is made up of single
digits corresponding to all but the last digit in the filename For
example an eBook of filename 10234 would be found at
```

Evaluate your program on the 5 datasets and measure (inside the program) the amount of data read from the input and the amount of (wall) time it took to clean all of the files. Make sure to clean the OS file system cache before you run an evaluation. Plot a diagram showing how the size of the datasets, measured in MiB, influences the throughput of your program, measured in MiB/second (datasets size divided by total amount of time to clean the dataset).

Answer the following questions:

1. What is the difference between the wall time and the CPU time? What method or function did you use to measure the wall time?
2. How big are Dataset 1 and Dataset 6, measured in MB and MiB (Megabytes and Mebibytes, respectively)?
3. How fast is your disk, measured in MiB/seconds? How does your program throughput fare in comparison to the speed of your disk and why?

4. Why would the dataset size influence the performance of your program on the virtual machine? What command did you use to clean the OS file system cache?

3 Word Count (20%)

Implement a program in C++ or in Java that receives as arguments an input directory and an output directory and that counts the number of unique words found in each file and writes them to the output directory.

The output files must follow the same folder structure as the input files. For example, if the program counts the words found in the input file stored at **CleanedDataset1/folder6/document265.txt**, it must store the counted words in the file at **CountedDataset1/folder6/document265.txt**, where **CleanedDataset1** was the input directory and **CountedDataset1** was the output directory.

In this program words are sequences of alphanumerical characters (0-9a-zA-Z) separated by a delimiter ('\', '\t', '\n', '\r\n', '\r'). **This program will use the output of the previous program as input.**

When the program finished counting the words from an input file it needs to write in the corresponding output file on each line the word and the number of occurrences, separated by a space.

For example, for the following input file:

```
EBooks posted since November 2003 with etext numbers OVER 10000 are
filed in a different way The year of a release date is no longer part
of the directory path The path is based on the etext number which is
identical to the filename The path to the file is made up of single
digits corresponding to all but the last digit in the filename For
example an eBook of filename 10234 would be found at
```

The program needs to create the corresponding output file that contains (this example includes only a subset of the output file):

```
...
filed 1
in 2
a 2
different 1
way 1
The 3
year 1
of 4
release 1
date 1
is 4
longer 1
part 1
the 6
directory 1
```

```
path 3
based 1
number 1
which 1
identical 1
to 3
filename 3
...
```

Evaluate your program on the 5 datasets and measure (inside the program) the amount of data read from the input and the amount of (wall) time it took to count the words of all files. Make sure to clean the OS file system cache before you run an evaluation. Plot a diagram showing how the size of the datasets, measured in MiB, influences the throughput of your program, measured in MiB/second (datasets size divided by total amount of time to count the words of the dataset).

Answer the following questions:

1. What data structure(s) did you use to implement the program and why?
2. What is the difference between compute-intensive, memory-intensive and IO-intensive applications?
3. Is your program compute-intensive, memory-intensive or IO-intensive and why?
4. Why would the dataset size influence the performance of your program on the virtual machine?

4 Sort Words (20%)

Implement a program in C++ or in Java that receives as arguments an input directory and an output directory and that sorts the words read from each file by frequency in descending order and writes the sorted words and their frequencies in a corresponding file in the output directory.

The output files must follow the same folder structure as the input files. For example, if the program sorts the words found in the input file stored at **CountedDataset1/folder6/document265.txt**, it must store the sorted words in the file at **SortedDataset1/folder6/document265.txt**, where **CountedDataset1** was the input directory and **SortedDataset1** was the output directory. **This program will use the output of the previous program as input.**

When the program finished counting the words from an input file it needs to write in the corresponding output file on each line the word and the number of occurrences, separated by a space, **similarly to the previous program**.

For example, for the following input file:

```
filed 1
in 2
a 2
different 1
```

```
way 1
The 3
year 1
of 4
release 1
date 1
is 4
longer 1
part 1
the 6
directory 1
path 3
based 1
number 1
which 1
identical 1
to 3
filename 3
```

The program needs to create the corresponding output file that contains:

```
the 6
of 4
is 4
The 3
path 3
to 3
filename 3
filed 1
in 2
a 2
different 1
way 1
year 1
release 1
date 1
longer 1
part 1
directory 1
based 1
number 1
which 1
identical 1
```

Evaluate your program on the 5 datasets and measure (inside the program) the number of words read from the input and the amount of (wall) time it took to sort the words in all files. Plot a diagram showing how the total number of words from the datasets influences the throughput of your program, measured in words/second (total number of words in the

datasets divided by total amount of time to sort the dataset).

Answer the following questions:

1. What data structure(s) did you use to implement the program and why?
2. What algorithm did you use to sort the data and why?
3. Is your program compute-intensive, memory-intensive or IO-intensive and why?
4. Why would the total number of words in a dataset influence the performance of your program on the virtual machine?

5 Where you will submit

You will have to submit your report to D2L and the code to a private git repository created for you for this programming assignment. The repository is created through GitHub Classroom and you will need to accept the assignment before you can clone it, by accessing this invitation link: https://classroom.github.com/a/T9GFMA2S. The first time you access the invitation link, the system will ask you to identify yourself. Please select the identifier that containts your school email address. After that you can clone the repository. Then you will have to add or update the source code and readme. Your submission on GitHub Classroom will not be graded unless you submit the report to D2L. The timestamp of your report on D2L will be used to determine if the submission is on time or late. If you cannot access your repository, please reach out at aorhean@depaul.edu.

6 What you will submit (10%)

As you work on the coding aspect of the assignment you should submit to the git repository each small functionality that you implement. When you have finished implementing the complete assignment you should submit your report to D2L. You need to hand in:

1. **Source code:** All of the source code, including comments and a CMake build file for a C++ solution, or a Maven build file for a Java solution.
2. **README.md:** A detailed manual describing the structure of your files and directory organization. The manual should contain a description of how to build the program and how to run the program.
3. **Report:** A written PDF document describing the overall assignment completion, along with screenshots and text answers to the assignment questions. Make sure to label the answers with the appropriate part and question number.

Do not upload the original/cleaned/counted/sorted data!

Submit step 1: code/build file/readme through git.
Submit step 2: report through D2L.

Grades for late submission will be lowered 20% per late day.